CEN 4072 – Software Testing

Spring 2026 | Group 15

Automated End-to-End Testing of GOG.com

Final Project Report

69	8	100%
Test Cases	Test Classes	Pass Rate

Team Members

Robert Benstine · Dennis Kerry · Myles Vinal

Instructor: Deepa Devasenapathy

GitHub: https://github.com/DennisKerry/GOG-Tester

1. Project Overview

1.1 Project Goal

The objective of this project is to design and implement a comprehensive automated end-to-end test suite for GOG.com, a major DRM-free PC gaming platform operated by CD Projekt. The suite validates the core user-facing functionality of the site through 69 automated test cases organized across 8 independent test classes.

The 8 test classes cover: home page structure, authentication, catalog search with interactive filters, game product pages, cart/checkout, community forums, URL-parameter-based catalog browsing, and the GOG Galaxy client download page.

By automating these tests, the team aimed to:

- Detect regressions in critical user flows (login, search, checkout) before they reach end users.
- Verify that GOG's Angular Single Page Application renders required UI elements correctly across sessions.
- Demonstrate the application of Selenium WebDriver with TestNG in a real-world, production environment.
- Practice handling automation challenges such as cross-origin iframes, GDPR consent modals, dynamic elements, and bot-detection headers.

1.2 System Under Test

Platform	GOG.com (Good Old Games) — https://www.gog.com
Operator	CD Projekt
Catalog	6,000+ DRM-free PC game titles
Architecture	React/Angular hybrid SPA; auth modal loads login.gog.com in a cross-origin iframe
Repository	https://github.com/DennisKerry/GOG-Tester

1.3 Test Classification & Architecture

All tests are classified as System / End-to-End tests. Each test class exercises a specific page or feature area of the GOG.com production site using a real Chrome browser controlled by Selenium WebDriver.

E2E Shared Browser — E2EBase

All 8 test classes extend E2EBase, which manages a single shared Chrome window for the entire suite run via @BeforeSuite / @AfterSuite. Each class picks up browser state left by the previous class, mirroring a real user journey:

- Primary E2E chain (steps 1–5): HomePageTest → LoginPageTest → SearchTest → GamePageTest → CartPageTest
- Extended E2E chain (steps 6–8): ForumTest → CatalogBrowseTest → GalaxyDownloadTest

E2EBase uses volatile boolean guard flags (suiteStarted / suiteStopped) to prevent double-initialization when TestNG discovers the @BeforeSuite annotation on each inheriting class. All @BeforeClass methods are annotated alwaysRun = true to ensure navigation happens even after a prior class failure.

Note: BaseTest (per-class driver) still exists and is used by AuthenticatedTest, but is not used by any active test class in the current run.

2. Test Suite

2.1 Suite Overview

The following table summarizes all 8 test classes, their functional coverage areas, and execution results. The full suite completes in approximately 3 minutes (observed: 183 seconds).

Test Class	Area Tested	Methods	Result
HomePageTest	GOG homepage: title, HTTPS, header, logo, Sign In button, Store link, game tiles, footer, cookies, legal links	11	11/11 ✓
LoginPageTest	GOG auth modal: iframe handling, email field, password field, submit	9	9/9 ✓

Test Class	Area Tested	Methods	Result
	button, forgot password, create account link		
SearchTest	Catalog page, search input, filters panel, discount filter, RPG genre filter, sort dropdown, search for 'witcher', tile count, Witcher 3 tile	10	10/10 ✓
GamePageTest	Witcher 3 product page: URL, HTTPS, title, H1, add-to-cart, price, description, screenshots, system requirements, DRM-free label, ratings	11	11/11 ✓
CartPageTest	Add to cart, cart counter, cart dropdown, game in dropdown, total price, Go to Checkout button, login prompt on checkout	7	7/7 ✓
ForumTest	Forum index: HTTPS, General Discussion link, game-specific forum links, Hot Topics, category thread listing, thread navigation, Cyberpunk 2077 forum	7	7/7 ✓
CatalogBrowseTest	URL-parameter catalog: base tile count, ?genres=rpg, ?order=asc:title sort verification, ?discounted=1 badges, ?query=cyberpunk, price display, combined params	7	7/7 ✓
GalaxyDownloadTest	GOG Galaxy page: HTTPS, heading, Windows download button, .exe href, macOS option, FAQ section, filesystem download verification	7	7/7 ✓
TOTAL	69	69/69 ✓	

2.2 Test Design Approach

Each test class extends E2EBase and operates on the shared browser instance. A @BeforeClass in each class navigates to the relevant page before tests run. The following design principles were applied throughout:

- CSS selectors, XPath, and WebDriverWait with a 20-second timeout are used for all element interactions.
- Shared helpers for cookie consent dismissal, age gate dismissal, page-load wait, and element presence checks are centralized in TestUtils.
- Each test within a class is independent — ordering within a class does not affect correctness.
- Anti-bot Chrome flags are applied globally via E2EBase to prevent automation detection.

3. Testing Workflow

The end-to-end testing pipeline follows the sequence described below. All steps are automated and triggered by a single mvn test command:

Step	Activity	Description
1	**Test Trigger**	mvn test or testng.xml run via Maven Surefire Plugin
2	**WebDriverManager**	Automatically downloads the matching ChromeDriver binary
3	**E2EBase.initDriver()**	@BeforeSuite — launches one headful Chrome with anti-bot flags; shared across all 8 test classes
4	**@BeforeClass**	Each class navigates to its starting URL; dismisses GDPR cookie consent
5	**Test Execution**	Each @Test method queries the DOM, asserts element presence / attribute values / URL state
6	**@AfterSuite**	E2EBase.quitDriver() closes the single shared browser
7	**Surefire Reports**	XML + HTML reports written to surefire-reports; custom HtmlProjectReportListener also writes project-report.html
8	**CI / Git**	All source committed to GitHub: https://github.com/DennisKerry/GOG-Tester

4. Work Conducted

4.1 Test Planning

The team selected GOG.com as the system under test because it is a publicly accessible, feature-rich e-commerce platform with well-documented automation challenges. The team identified 8 functional areas and mapped each to a dedicated test class. Key selection criteria included:

- Publicly accessible without authentication for most test flows.
- Cross-origin iframe authentication layer presenting non-trivial automation complexity.
- Active GDPR consent modal and age gate requiring automated dismissal handling.
- Aggressive bot-detection headers requiring ChromeDriver configuration.
- CDN-served binary downloads requiring Chrome DevTools Protocol for filesystem verification.

4.2 Key Technical Challenges

E2E Shared Browser

All 8 test classes share one Chrome instance via E2EBase. Guard flags prevent double @BeforeSuite initialization. Each class's @BeforeClass is annotated alwaysRun = true to ensure navigation happens even if a prior class had a failure.

Cross-Origin Login Iframe

The GOG Sign In modal loads login.gog.com inside an iframe. LoginPageTest polls the frame list until it finds the one containing #login_username, then switches into it for all structural assertions.

React-Rendered Text

WebElement.getText() returns empty strings for React-rendered elements in some page configurations. The solution was to use getAttribute("textContent") combined with visibilityOfElementLocated() to reliably retrieve visible text.

GOG Galaxy CDN Download Verification

GalaxyDownloadTest uses the Chrome DevTools Protocol (Browser.setDownloadBehavior) to redirect downloads to target/galaxy-download-test/, then clicks the download button and polls the filesystem for up to 20 seconds for a .exe or .crdownload file to appear, confirming the CDN is actually serving the installer.

4.3 Integration & Execution

All 8 test classes are integrated in testng.xml, which defines the suite name "GOG Selenium Test Suite" and runs each class in sequence via the shared E2EBase browser. Tests are executed via mvn test, with Maven Surefire Plugin 3.2.5 generating per-class XML reports in target/surefire-reports/. A custom HtmlProjectReportListener also writes project-report.html.

Final execution result: 69 tests run · 0 failures · 0 errors · 0 skipped. Total suite runtime: approximately 3 minutes (observed: 183 seconds).

5. Individual Contributions

Team Member	Contributions
Robert Benstine	Authored HomePageTest and CatalogBrowseTest; configured testng.xml integration suite; contributed to BaseTest/E2EBase shared infrastructure; report writing.
Dennis Kerry	Authored LoginPageTest and CartPageTest (including cross-origin iframe and dynamic cart navigation); authored GalaxyDownloadTest (CDP filesystem download verification); GitHub repository setup and maintenance; debugging and final test fixes.
Myles Vinal	Authored SearchTest, GamePageTest, and ForumTest; implemented React text content workaround using getAttribute("textContent"); PowerPoint presentation authoring; report writing.

6. Tools & Technologies

Tool / Technology	Version	Purpose
Java (JDK)	25.0.2 (LTS)	Primary programming language for all test code
Selenium WebDriver	4.18.1	Browser automation — finding elements, clicking, asserting
TestNG	7.9.0	Test framework — annotations, assertions, suite runner
Apache Maven	3.9.12	Build tool and dependency manager; runs tests via Surefire 3.2.5
WebDriverManager (bonigarcia)	5.7.0	Automatically downloads the correct ChromeDriver binary
Google Chrome	147.0.7727.56	Browser under test
ChromeDriver	147.0.7727.56	WebDriver implementation for Chrome
Git / GitHub	—	Version control and code hosting
Windows 11	—	Operating system used for test execution
IntelliJ IDEA / VS Code	—	IDE used for development

7. Motivation & Quality Impact

7.1 Why Automate?

GOG.com serves millions of users globally. Regressions in core flows — login, search, or checkout — can directly impact revenue and user trust. Manual testing at scale is slow and error-prone; an automated suite provides repeatable, objective verification of critical functionality. The suite runs in approximately 3 minutes and produces a structured pass/fail report, compared to hours of manual effort.

7.2 Issues the Test Suite Prevents

- **Broken authentication: LoginPageTest verifies the sign-in iframe, email/password fields, submit button, and create account link. A frontend deployment breaking any of these would be immediately flagged.**
- **Search regressions: SearchTest confirms search returns game tiles with titles and cover images, and that filters and sort controls function correctly.**
- **Cart/checkout unavailability: CartPageTest verifies the cart counter, dropdown, line items, total price, and checkout flow including the guest login prompt.**
- **Game page content missing: GamePageTest checks the buy button, price, description, screenshots, system requirements, DRM-free label, and ratings.**
- **Forum accessibility: ForumTest ensures community-facing forum pages, thread listings, and game-specific forums are reachable.**
- **Catalog filter regressions: CatalogBrowseTest verifies URL-parameter-driven filtering (genre, sort, discount, search query, combined params) works correctly.**
- **Galaxy client CDN broken: GalaxyDownloadTest physically downloads the installer and verifies the file appears on the filesystem.**

7.3 How the Suite Improves System Quality

The tests serve as a regression safety net and executable specification. After any GOG frontend update, running mvn test produces a pass/fail report for all 69 test cases within approximately 3 minutes. The shared-browser E2E chain mirrors a real user journey, catching stateful regressions that isolated unit tests cannot detect.

8. Appendix

Appendix A — Automation Constraints & Solutions

Constraint	Solution Applied
Shared browser across 8 classes	E2EBase @BeforeSuite/@AfterSuite with volatile boolean guard flags; all @BeforeClass annotated alwaysRun = true
Cross-origin login iframe (login.gog.com)	LoginPageTest polls driver.findElements(By.tagName("iframe")), switches into the frame containing #login_username
React-rendered text – getText() returns empty string	Used getAttribute("textContent") and visibilityOfElementLocated()
GDPR cookie consent modal intercepts clicks	TestUtils.dismissCookieConsent() dismisses with JS click fallback
GOG bot-detection headers	ChromeOptions: --disable-blink-features=AutomationControlled; excludeSwitches: enable-automation; useAutomationExtension: false
Age gate on mature game pages	dismissAgeGate() helper clicks the first button inside [class*='age-gate'] via JavaScript click
Galaxy installer download CDN verification	Chrome DevTools Protocol Browser.setDownloadBehavior redirects download to target/galaxy-download-test/; filesystem polled for .exe/.crdownload

Appendix B — Project Structure

```
src/test/java/com/gog/
├── base/
│   ├── E2EBase.java            (@BeforeSuite/@AfterSuite shared driver, guard flags)
│   └── BaseTest.java           (per-class driver; used by AuthenticatedTest only)
├── tests/
│   ├── HomePageTest.java       (11 tests)
│   ├── LoginPageTest.java      (9 tests)
│   ├── SearchTest.java         (10 tests)
│   ├── GamePageTest.java       (11 tests)
│   ├── CartPageTest.java       (7 tests)
│   ├── ForumTest.java          (7 tests)
│   ├── CatalogBrowseTest.java  (7 tests)
│   └── GalaxyDownloadTest.java (7 tests)
└── utils/
    └── TestUtils.java          (dismissCookieConsent, dismissAgeGate, performLogin,
waitForPageLoad, isElementPresent)

testng.xml                      (full suite integration - 8 test classes, parallel=none,
shared E2EBase driver)
pom.xml                         (Maven build, Selenium 4.18.1, TestNG 7.9.0, Surefire
3.2.5)
```

Appendix C — Test Case Details

HomePageTest (11 tests)

- Verify the GOG home-page title contains 'GOG'
- Verify the home page is served over HTTPS
- Verify a header navigation bar is present
- Verify the GOG logo is present in the header
- Verify the Sign In button is present
- Verify the Store navigation link is present in the header
- Verify at least one game product tile is rendered on the home page
- Verify a footer element is rendered on the home page
- Verify that session cookies are set after visiting GOG.com
- Verify the home page renders more than 3 game product tiles (catalog richness)
- Verify the footer contains legal or informational links (privacy, terms, cookies, or refund)

LoginPageTest (9 tests)

- Verify the login popup/form is visible after clicking Sign In
- Verify the email/username field is present in the login form
- Verify the email field accepts text input (type email or text)
- Verify the password field is present in the login form
- Verify the password field has type=password so input is masked
- Verify the Log in now submit button is present and enabled
- Verify a Forgot Password/Password Reset link is present
- Verify a Sign Up/Create Account link is present in the popup
- Verify the login form is structurally complete and dismiss the popup

SearchTest (10 tests)

- Verify the browser is on the GOG games catalog after clicking Store
- Verify the search input is present on the catalog page
- Verify the filters panel is present on the full games catalog
- Verify 'Show only discounted' filter narrows the catalog and changes tile count or URL
- Verify the RPG genre filter narrows catalog results and reflects in the URL
- Verify the sort dropdown reorders catalog results and updates the URL with the sort parameter
- Search for 'witcher' in the catalog search box
- Verify at least one product tile appears after searching 'witcher'
- Verify The Witcher 3: Wild Hunt tile is visible in the results
- Verify The Witcher 3 tile is present and ready to click

GamePageTest (11 tests)

- Verify the browser navigated to The Witcher 3 page on gog.com
- Verify the game page is served over HTTPS
- Verify the page title contains 'Witcher'
- Verify an H1 game-title heading is visible on the product page

- Verify the Add to Cart button is present
- Verify a price or ownership indicator is visible
- Verify the game description or overview section is present
- Verify at least one screenshot or media image is present
- Verify the system requirements section is present
- Verify a DRM-free label or protection indicator is present (GOG's core brand promise)
- Verify a user rating or community review section is present

CartPageTest (7 tests)

- Verify the Add to Cart button exists or the game is already in cart
- Verify the cart counter in the header shows at least 1 item
- Verify the cart dropdown opens by clicking the cart icon in the header
- Verify The Witcher 3: Wild Hunt is listed in the cart dropdown
- Verify a total price is displayed in the cart dropdown
- Verify the 'Go to checkout' button is present in the cart dropdown
- Verify clicking 'Go to checkout' shows a login prompt for guest users

ForumTest (7 tests)

- Verify the GOG forum index page loads over HTTPS with a non-empty title
- Verify the General Forums section contains a link to the General Discussion category
- Verify game-specific forum links (Witcher 3 or Cyberpunk 2077) appear on the forum index
- Verify the Hot Topics sidebar shows active community thread links
- Verify navigating to the General Discussion category URL returns a thread listing
- Verify clicking a forum thread opens a page with a non-empty title and stays on gog.com
- Verify the Cyberpunk 2077 game-specific forum page is directly accessible and has content

CatalogBrowseTest (7 tests)

- Verify the base GOG catalog page displays at least 20 game tiles
- Verify ?genres=rpg URL param filters catalog to RPG titles and retains param in URL
- Verify ?order=asc:title sorts the catalog so the URL reflects the sort and tiles are in A-Z order
- Verify ?discounted=1 returns discounted games with visible discount badges
- Verify ?query=cyberpunk returns Cyberpunk 2077 as a result and reflects query in URL
- Verify catalog game tiles display price information confirming active store functionality
- Verify combined ?genres=rpg&order=asc:title params both appear in URL and return game results

GalaxyDownloadTest (7 tests)

- Verify the GOG Galaxy landing page loads over HTTPS with a title referencing Galaxy or GOG
- Verify the 'GOG GALAXY 2.0' product heading is visible on the page

- Verify a Windows installer download button is present on the Galaxy page
- Verify the Windows download link href references a .exe or GOG_Galaxy installer file
- Verify a macOS download option (.pkg or macOS text link) is present on the Galaxy page
- Verify the FAQ section exists with at least one question on the Galaxy page
- Verify clicking the Windows download button causes the installer file to appear on the local filesystem

Appendix D — References

- Selenium WebDriver Documentation — https://www.selenium.dev/documentation/
- TestNG Documentation — https://testng.org/doc/
- WebDriverManager (bonigarcia) — https://bonigarcia.dev/webdrivermanager/
- Apache Maven Documentation — https://maven.apache.org/guides/
- GOG.com System Under Test — https://www.gog.com
- GitHub Repository — https://github.com/DennisKerry/GOG-Tester